EC2 (Elastic Compute Cloud)

EC2 is a web service provided by Amazon Web Services that allows users to create and manage virtual servers in the cloud. These virtual servers are called instances.EC2 helps users run applications without buying or maintaining physical servers.

EC2 INSTANCE TYPES

Pricing Models

In Amazon Web Services, pricing models define how you are charged for EC2 instances.

1. On-Demand

- Pay per hour or second
- No long-term commitment
- Best for testing, learning, short-term use

2. Reserved Instances

- Commit for **1 or 3 years**
- Lower cost than On-Demand
- Best for long-running applications

3. Spot Instances

- Uses unused AWS capacity
- Very low cost
- Instance can be stopped anytime by AWS

4. Dedicated Instances

- Runs on **dedicated physical hardware**
- Higher cost
- Used for compliance and security needs

EC2 CONFIGURATION

Instance configuration defines **performance type** (CPU, memory, storage).

1. Compute Optimized (c)

- High CPU performance
- Used for compute-intensive tasks

Example: c5, c6i

2. Memory Optimized (r, x2)

- High RAM capacity
- Used for databases and in-memory apps

Example: r5, x2idn

3. Storage Optimized (i, d, h)

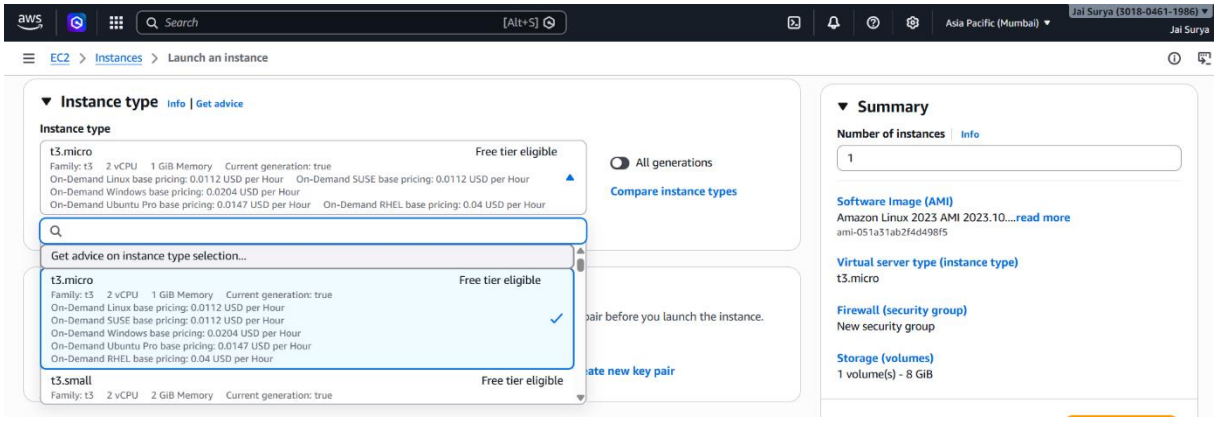
- High disk performance
- Used for big data and data processing

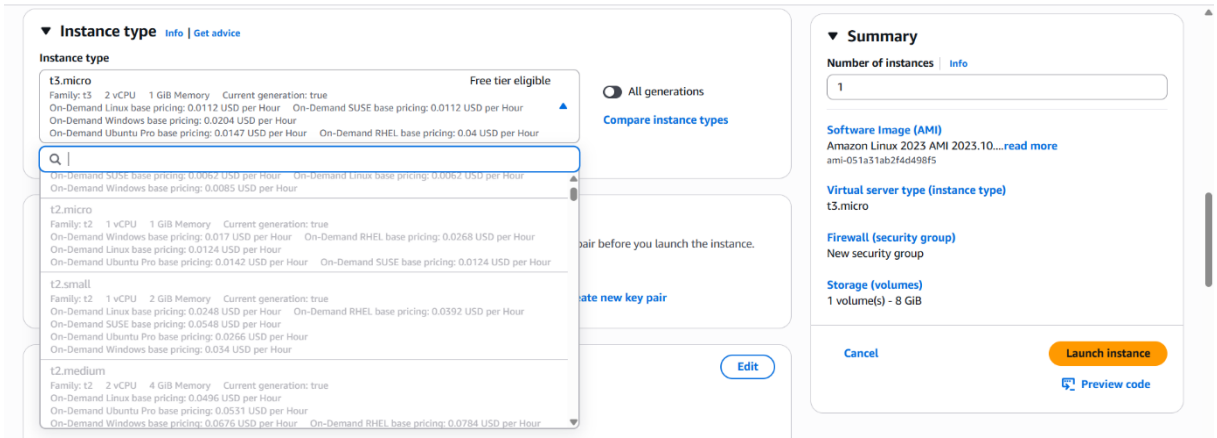
Example: i3, d2, h1

4. General Purpose (t, m)

- Balanced CPU and memory
- Used for most applications

Example: t2.micro, m5





KEY PAIR IN EC2

A **Key Pair** is used to securely connect to an EC2 instance.

It consists of **two keys**:

1. .pem File (Private Key)

- Downloaded while creating the EC2 instance
- Used to connect to the instance using SSH
- Must be kept **secure** and **not shared**

Example:

For external connecting (gitbash)

ssh -i keypair-name.pem ec2-user@<Public-IP>

2. .ssh Folder (Public Key)

- Located inside the Linux home directory
- Contains the **authorized_keys** file
- Stores the **public key** used for authentication

Path: ~/.ssh/authorized_keys

HIDDEN FILES AND FOLDERS

- Files or folders starting with a **dot (.)** are **hidden** in Linux
- Example: .ssh, .bashrc

To view hidden files:

ls -a or ls -la

SECURITY CREDENTIALS (AWS)

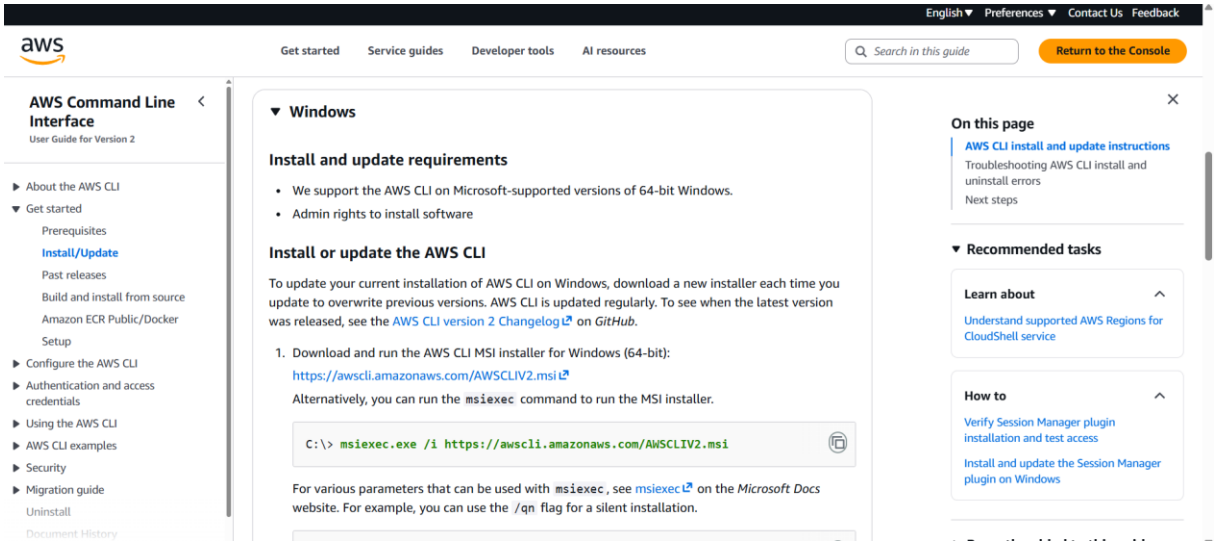
Download AWS CLI for Windows

Open your browser

Go to: https://aws.amazon.com/cli/

Click Download for Windows (64-bit)

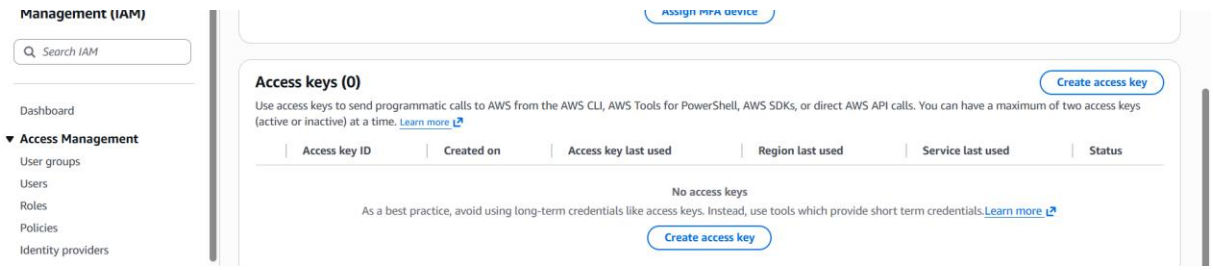
Save the AWSCLIV2.msi file to your computer

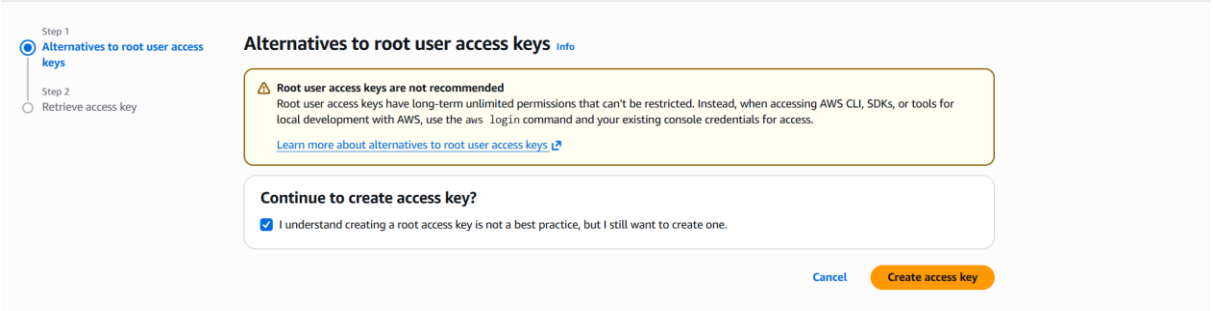


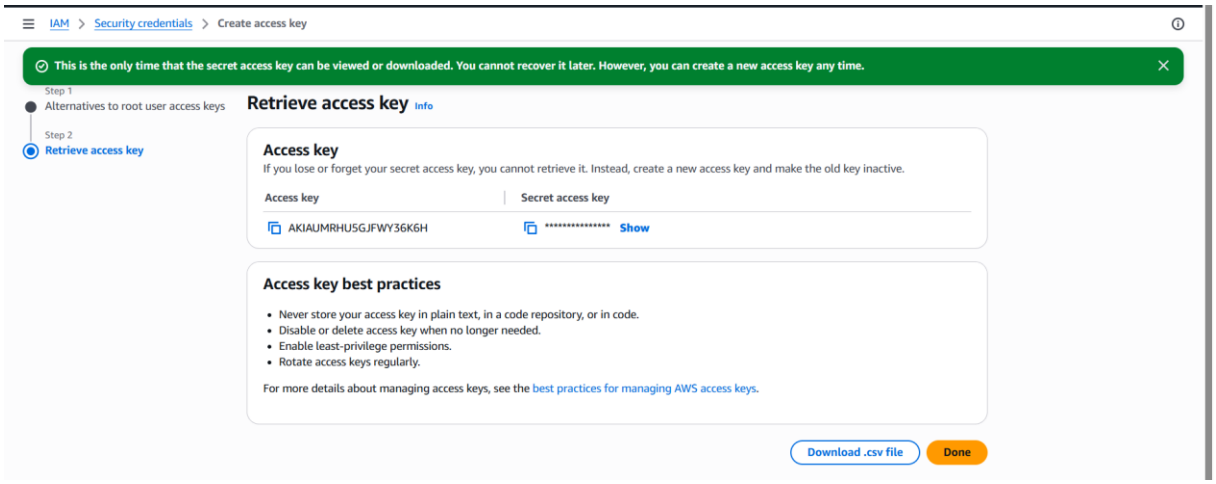
Go to Security Credentials & Create Access Keys

1. Login to AWS Management Console
2. Click Account name (top-right corner)
3. Select Security credentials
4. Scroll to Access keys section
5. Click Create access key
6. Choose CLI as use case
7. Click Create access key
8. copy:

- Access Key ID
- Secret Access Key







Configure AWS CLI Using Git Bash

Open Git Bash

Run the command:

aws configure

Enter:

- AWS Access Key ID
- AWS Secret Access Key
- Default region name (example: ap-south-1)
- Default output format (json)
- Credentials are saved securely in:

~/.aws/credentials

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~
$ aws configure
AWS Access Key ID [****************6K6H]: AKIAUMRHU5GJNHVRXKHG
AWS Secret Access Key [****************cpHP]: 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
Default region name [None]:
Default output format [None]:

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~
$
```

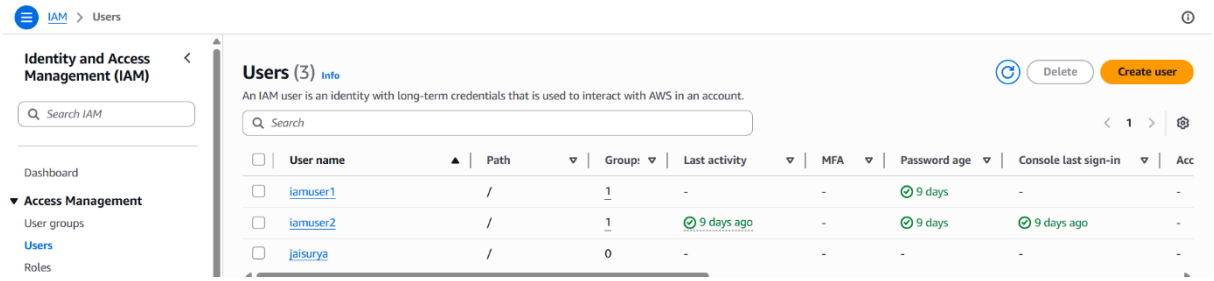
Create IAM User in cli

aws iam create-user --user-name jaisurya

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~
$ aws iam create-user --user-name jaisurya
{
    "User": {
        "Path": "/",
        "UserName": "jaisurya",
        "UserId": "AIDAUMRHU5GJH3CZDLYIH",
        "Arn": "arn:aws:iam::301804611986:user/jaisurya",
        "CreateDate": "2026-03-05T03:03:01+00:00"
    }
}
```

Verify IAM User in AWS Console

1. Login to AWS Management Console
2. In the search bar, type IAM
3. Open IAM (Identity and Access Management)
4. Click Users from the left menu
5. Check the list of users
6. Verify the newly created IAM user



Delete AWS CLI Credentials (Before Reconfiguration)

Steps (Git Bash):

- Check hidden AWS folder:

ls -la | grep .aws

- Move into .aws directory:

cd .aws

- List files inside .aws:

ls

- View existing credentials:

cat credentials

- Delete credentials file:

rm -rf credentials

- Verify deletion:

ls

If credentials file is not listed → credentials deleted successfully

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~
$ ls -la | grep .aws
drwxr-xr-x 1 JAISURYA 197121        0 Mar  5 07:58 .aws/

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~
$ cd .aws

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/.aws
$ ls
config  credentials

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/.aws
$ cat c
config       credentials

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/.aws
$ cat credentials
[default]
aws_access_key_id = AKIAUMRHU5GJNHVRXKHG
aws_secret_access_key = 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

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/.aws
$ rm -rf credentials

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/.aws
$ ls
config

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/.aws
$
```

STEPS TO CREATE AND LIST S3 BUCKET (AWS CLI – GIT BASH)

Step 1: Create S3 Bucket

aws s3 mb s3://myfirstbucket73584205 --region ap-south-1

Creates an S3 bucket in **Mumbai region**

Step 2: List S3 Buckets

aws s3 ls

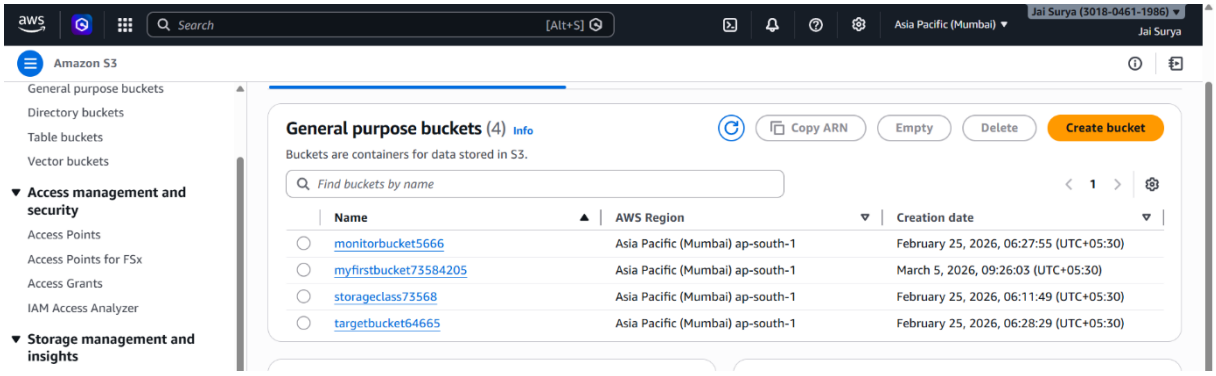
Displays all S3 buckets in your AWS account

```
make_bucket failed: s3://myfirstbucket73584205 An error occurred (InvalidLocationConstraint) when calling

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/.aws
$ aws s3 mb s3://myfirstbucket73584205 --region ap-south-1
make_bucket: myfirstbucket73584205

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/.aws
$ aws s3 ls
2026-02-25 06:27:55 monitorbucket5666
2026-03-05 09:26:03 myfirstbucket73584205
2026-02-25 06:11:49 storageclass73568
2026-02-25 06:28:29 targetbucket64665

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/.aws
$
```

Verify S3 Bucket in AWS Console

- Login to the Amazon Web Services Management Console.
- Open the Amazon S3 service using the search bar.
- Go to the Buckets section.
- Check the bucket name myfirstbucket73584205 in the list.